How We Build

Our software development process, start to finish

A working handbook, not a policy document. If a step here does not make sense on a real project, the step is wrong and we change it.

1. The idea in one page

AI writes most of our code now. That changed where the difficulty sits. Writing code used to be the slow part; now it is fast. What stayed slow is deciding what to build, and checking that what came back is actually right.

So this process spends most of its effort at the front and the back of a project, and comparatively little in the middle where the code gets written.

The problem with how we worked before

We used to give the client brief to Claude, ask it what was missing, get a plan back, and start building. That is quick to start and it has one serious flaw: Claude was checking the requirements it had just inferred from the same brief. It cannot tell you what the client did not say.

Everything downstream inherited that gap. There was also nothing between "start building" and "ship" — no point where a person checked the work, and no agreed definition of what finished meant.

What we do instead

Stage	What happens
Understand	A person asks the client the questions Claude found. Not Claude answering them.
Agree	We write a specification the client reads and signs.
Design	We map how the features relate before choosing a stack.
Contract	We write the tests first and freeze them.
Build	The agent implements against those frozen tests.
Verify	A separate agent reviews it. Then a person reads the findings.
Hand over	Someone who did not build it runs it from the README.
Improve	What went wrong becomes a rule for next time.

Three ideas worth understanding

Tests are written first, then frozen

Before any code is written, we turn the acceptance criteria into tests and commit them. From that point nobody edits them — not us, and specifically not the coding agent.

An agent that can edit its own tests will edit them. Not maliciously; it loosens an assertion to get past a stubborn failure, and the change looks perfectly reasonable in a diff. Then the suite is green and the bug ships.

Frozen tests are also a commercial boundary. If a client request means changing the tests, that is new scope, and it becomes a priced conversation instead of absorbed work.

A second agent reviews the work

The agent that wrote the code is a poor judge of whether it is correct, because it will find the same reasoning that made it write the code that way. So review runs in a completely fresh session with no memory of the implementation, given only the specification, the tests, and the diff.

It reports findings. It never gives a verdict. A green stamp from a reviewing agent would just become a reason to skip human review, which would leave us worse off than before.

Every gate is an artifact, not a meeting

Gates that require someone to remember them get skipped. Every gate here is a document that exists or does not, or a check that CI runs automatically. That is why they hold.

2. Before you start

Pick a track

Decided when the brief arrives, in writing. Not revisited casually — changing track is a commercial conversation.

Track	When	Phases
P — Prototype	Pitch, demo, spike, internal exploration	0, 1, 5
L — Light	Under 10 days, one area, we maintain it	0, 1, 4, 5, 6, 7
F — Full	Everything else	All of them

> **Prototypes never ship**
>
> Track P work lives on a branch named prototype/ and never reaches a client environment. If the client wants it, we rebuild it properly. A prototype that ships by accident becomes a maintenance liability nobody priced.

Track L becomes Track F automatically if a second area appears, an external integration is added, the client's own team will maintain it, or the build passes 15 days. That escalation is a change request, not a quiet decision.

The two places you work

Where	Phases	What it is for
Claude Project	0 to 2	Thinking. Questions, specification, system model, estimate, stack.
Claude Code	3 to 8	Building. Anything that touches the repository.

One Claude Project per client. Attach these four files at the start:

- 00-START-HERE.md — the setup guide
- 01-prompt-library.md — every phase prompt
- 02-process-additions.md — change control, estimation, post-launch
- 03-artifact-templates.md — blank structures for what you produce

Then add the client brief when it arrives.

> **Do not attach the repository files to the Project**
>
> AGENTS.md, the security baseline, and the other docs/ files are read by Claude Code from disk. Putting them in the Project gives the false impression they are covered when Claude Code has not actually read them.

3. Phases 0 to 2 — in the Claude Project

Phase 0 — Intake

Paste the brief into the Project and run the gap analysis prompt. It gives you questions, not a specification.

Take the blocking questions to the client on a call. Write the answers down the same day — recall degrades fast and these answers are the basis of everything downstream.

Run the dependency extraction prompt at the same time. It lists everything you need from the client: API keys, brand assets, server access, copy. Request all of it now, not when it becomes blocking.

Gate: no blocking question left unanswered.

> **Why a person asks the questions**
>
> This is the fix for the flaw in how we used to work. Claude is good at spotting what a brief does not say. It cannot tell you what the client meant, or which stated requirement is commercially dangerous. Only a conversation does that.

Phase 1 — Specification

Run the spec prompt with the brief and the intake answers. You get a draft in two layers: Part A that the client reads, Part B for us.

Then edit it. This step cannot be skipped. The draft will contain plausible inference presented as fact, and finding that is the job. It reads well, which is exactly the problem.

Send Part A to the client, walk them through it on a call, get a signature.

Gate: signed specification, with the out-of-scope section included.

Two sections earn their keep repeatedly:

- Explicitly out of scope — name things specifically. "No mobile app, no Salesforce sync, no multi-currency" settles arguments. "No advanced features" settles nothing.
- Assumptions — every question the client did not answer, with what we assumed and what it costs if we are wrong. This is what makes the signature mean something.

Phase 1.5 — System model

Run the system model prompt on the signed spec. It produces four things: a map of which area owns what, a grid of which features read and write which data, a dependency graph, and a list of concerns that cut across everything.

> **The failure this prevents**
>
> A weekly review page and a KPI dashboard look like two separate features. They are not — they both display the same numbers. Built separately, they compute those numbers separately, and eventually they disagree. Fixing that in production is a rewrite. Catching it here costs an hour.

Review the read/write grid yourself. A wrong call about which area owns a piece of data looks entirely reasonable on the page and surfaces months later as two screens showing different figures.

Phase 1.6 — Estimate

Size each capability from the system model as small, medium, or large, then convert to days. Add the fixed phase costs, apply multipliers, add contingency.

If something is too big to size, it is not a size — it means Phase 1 was not detailed enough for that piece. Split it.

Record the estimate against the actual at project close. After five projects the multipliers become ours instead of borrowed, and that is the single highest-value thing we can do for margin.

Phase 2 — Architecture and stack

Ask for three stack options with trade-offs. Then decide yourself and write the decision record.

The model can lay out options. It cannot weigh the things that actually decide this: who maintains the code after handoff and what they can debug, what the client will pay to host, how long we are on the hook.

Gate: a named person signs the decision record. Not the model.

The record also covers the operational decisions that otherwise get made by accident at handover: migration tooling, environments, backups, error tracking, and who pays for all of it afterwards.

4. Moving to the repository

You now have five documents. Move them into a fresh repository.

1. Create a repository from the agency template on GitHub.
2. Clone it and open the folder.
3. Copy the five documents into the docs/ folder.
4. Commit and push.

Document	From
01-spec.md	Phase 1
02-system-model.md	Phase 1.5
03-adr.md	Phase 2
04-estimate.md	Phase 1.6
05-dependencies.md	Phase 0, kept updated

> **This step is not optional**
>
> Claude Code reads these files from disk. If the specification is not in the folder, Claude Code is working from whatever you paste into the chat, which is a summary rather than the document.

5. Phases 3 to 8 — in Claude Code

Phase 3 — Set up the harness

Run one command. It reads the decision record, works out the stack, installs the right test and build scripts, and switches on the CI checks.

```
./scripts/init-stack.sh
```

If it says the decision record is ambiguous — a project using two languages, say — tell it which one:

```
./scripts/init-stack.sh python
```

Then paste the Phase 3 prompt. It fills in AGENTS.md, the rules file every coding agent reads on this project.

Push, and check the Actions tab on GitHub shows green.

> **Stop until CI is green**
>
> You are checking the alarm system works before there is anything to protect. Skip this, and when a test fails in three weeks you will not know whether the code is broken or the pipeline was never wired up correctly.

Phase 4 — Write and freeze the tests

Paste the Phase 4 prompt. It writes tests into three folders:

Folder	Contains	Agent sees it while building?
tests/working	The normal path for every requirement	Yes
tests/holdout	Empty states, edge cases, permissions, concurrency	No
tests/cross-feature	The same number is identical everywhere it appears	No

Check the coverage table it produces — every acceptance criterion should have tests, with more depth on anything involving money, permissions, or deletion.

Commit. They are frozen now, and CI blocks any change to them from this point on.

Have the client sign the scenario list too. That is what converts a later scope argument into a priced change request.

Phase 5 — Build

Paste the Phase 5 prompt once per feature, working in the dependency order from the system model. Start a fresh session for each feature to keep the context clean.

The agent builds against the frozen tests and iterates until they pass, capped at five attempts on the same failure.

> **If the agent says a test looks wrong, stop**
>
> Do not let it change the test. A test that seems wrong almost always means the requirement was ambiguous, and that is a decision for a person and probably a conversation with the client.

Deploy each finished feature to staging. You and the client should be clicking through a real application well before the end.

Phase 6 — Independent verification

Close the session and open a completely new one. This is the step that stops working if you get it wrong.

Give the fresh session the specification, the test scenarios, and the diff — nothing else. No implementation history, no reasoning about why the code was written that way.

Run two separate passes: the general verification prompt, then the security prompt from docs/security-baseline.md. Security review and correctness review do not share attention well.

You get back structured findings: where the code diverges from the spec, requirements with no implementation, any test file that was touched, hallucinated dependencies, missing authorisation checks, and anything built that nobody asked for.

Then a person reads the findings and the test diff. Two minutes. That is the whole human cost of this gate, which is why it actually gets done.

Phase 7 — Handover

Paste the Phase 7 prompt. It writes the README, the operations runbook, the environment variables document, and the known limitations.

Then test it properly: clone the repository into a fresh folder and follow your own README. If you cannot get it running without knowing something that is not written down, it is not finished.

The client also receives the completed security checklist and the frozen scenario list as the record of what was agreed.

Phase 8 — Improve the template

Fifteen minutes at project close. Paste the Phase 8 prompt with notes on what went wrong during the build.

It sorts problems into three groups: preventable by a rule, specific to this project, or not preventable at all. The rules go into the master template, so the next project starts better than this one did.

This is the step everyone skips, and it is the one that makes the tenth project cheaper than the third.

6. Things that run throughout

Change requests

A client asks for something new in week three. This is the most common way a profitable project becomes an unprofitable one, so it gets a defined path.

It is a…	When	What happens
Bug	Contradicts something in the signed spec	We fix it, no charge
Change	Not traceable to a signed acceptance criterion	Assessed, priced, approved in writing first

The test: if it means changing the frozen tests, it is a change. If it makes the existing tests pass, it is a bug. This is why freezing the tests matters commercially and not just technically.

Every change gets written up before any work starts, including when we intend to do it for free. An unrecorded favour is invisible when the relationship goes wrong.

Run the impact assessment prompt — it walks the dependency graph and finds the consequences that are not obvious, like a small-looking addition touching something three finished features already depend on.

Client dependencies

Waiting on the client is the most common cause of overrun and the easiest to lose track of. The register lists everything we need from them, with dates.

When something is more than five days late, send a written note stating the new delivery date as a fact. Not a complaint — just the arithmetic, at the time it happens rather than at the end.

When the client asks why the date moved, the answer is a table with dates in it.

After delivery

The build ends at Phase 8. The relationship does not.

Everything that comes in gets classified within one working day: bug, change, environment problem, or support question. Bugs inside the warranty period are fixed free; everything else is a change request.

Every bug fix starts with a test. Write the test that should have caught it, commit it on its own while it is still failing — that is the reproduction — then fix it. The suite gets stronger with every defect, which is what makes maintenance get cheaper rather than more expensive.

7. Quick reference

The whole thing

Phase	Where	You do	Gate
0 Intake	Project	Ask the client the questions	Nothing blocking left open
1 Spec	Project	Edit the draft properly	Client signs
1.5 Model	Project	Review the read/write grid	No ambiguities left
1.6 Estimate	Project	Size the capabilities	Sanity check
2 Stack	Project	Decide and sign	A named person signs
3 Harness	Code	Run init-stack, fill AGENTS.md	CI green
4 Tests	Code	Check coverage, commit	Frozen, client signs
5 Build	Code	One feature at a time	Tests pass, none edited
6 Verify	Code	Fresh session, read findings	Blocking items resolved
7 Handover	Code	Run your own README	A stranger can run it
8 Improve	Code	Feed rules back	Template updated

Never do these

- Edit a test to make it pass
- Run a migration against any environment — generate the file, a person reviews it
- Add a dependency that is not in the decision record
- Skip authorisation on an endpoint because it is "internal"
- Trust a user ID, role, or tenant that came from the request body
- Build something a client asked for verbally, without a change request
- Let a prototype reach a client environment
- Run Phase 6 in the same session as Phase 5

The four places a person is essential

Most of this process is agent work inside gates. These four are not, and skipping any of them removes the reason the rest works.

Phase	Why it has to be you
0 — asking the client	A model checking requirements it inferred from the same brief tells you nothing
1 — editing the spec	The draft contains inference presented as fact, and it reads well
1.5 — the read/write grid	A wrong ownership call looks reasonable and surfaces months later
2 — the stack decision	Trade-offs against business context the model cannot weigh

If you remember nothing else

> **Three things**
>
> CI green before you write any code. A fresh session for Phase 6. Tests frozen once committed. Everything else you can be loose about — those three are the process.

And the habit that matters most: when something is ambiguous, stop and ask. The expensive failures all come from someone — person or agent — quietly guessing and being wrong. Asking costs ten minutes. Guessing costs a week, and you find out in week five.

8. First time through

Run the whole process on one small, real, low-risk client project before using it everywhere. Not the biggest one, and not an internal project — internal work does not apply real pressure to the gates.

Keep a friction log while you go: one line every time the process annoys someone, noting which phase. Fix nothing in the middle of the project.

At the end, that log is your Phase 8 input, and it is worth more than any further planning. Expect two or three phases to be wrong in ways nobody predicted. Fix those, then roll it out properly.

> **Why not roll it out everywhere at once**
>
> That is the standard way a process like this dies. The team finds it heavy, quietly skips steps, and you end up worse off than before — because now there is a false sense of rigour on top of the same gaps.